



Python应用基础

第10章 数据库应用

陈刚

中山大学岭南学院

提高篇目录

Contents

第 8 章

面向对象编程

第 9 章

异常处理

第 10 章

数据库应用

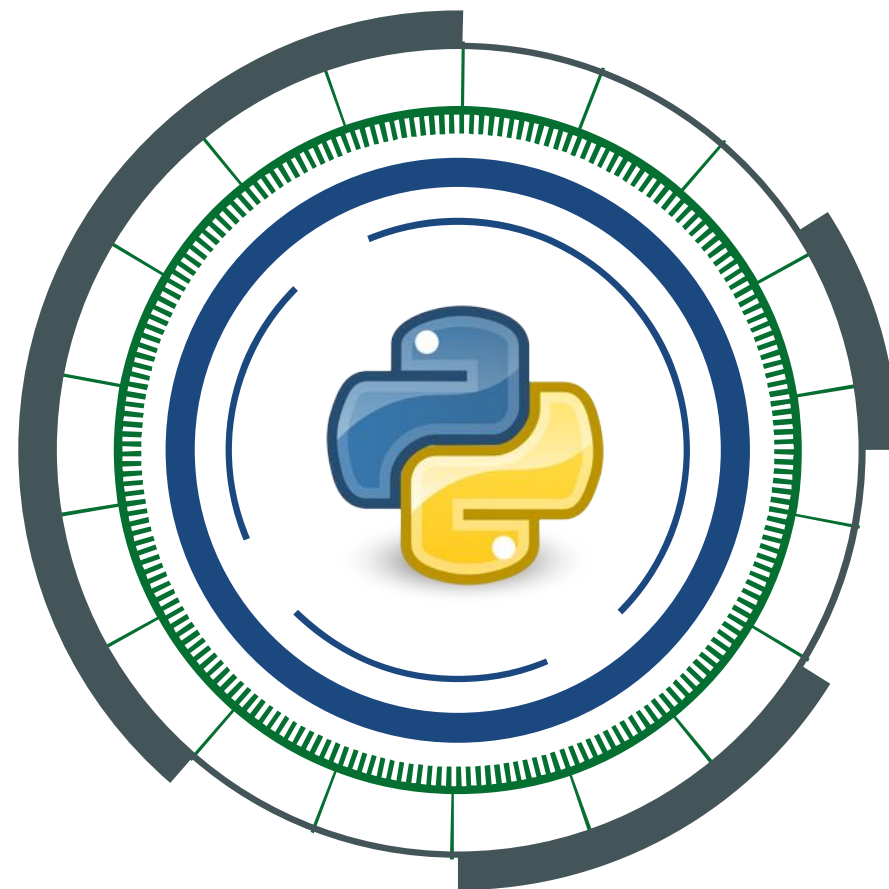


pythonTM
programming



学习目标

- 1 了解数据库的概念及其分类
- 2 掌握 SQLite 数据库的使用
- 3 熟悉 MySQL、SQL Server、Oracle、MongoDB 数据库的使用
- 4 灵活运用数据库解决实际问题





10.1

数据库简介

数据库简介

- 数据库 (database) 是按照 **数据结构** 来 **组织、存储和管理** 数据仓库的，是一种"电子化的文件柜"。
- 数据库不仅支持数据在计算机内的长期存储，而且支持跨平台、跨地域的数据操作，是有组织、可共享、统一管理的大量数据的集合。
- 当前数据库分为 **关系型数据库** 和 **非关系型数据库**。

关系型数据库

- 关系型数据库把复杂的数据结构抽象为简单的二元关系，通过二维表格关系模型来组织数据。
- 在关系数据库中，每一行被称为一条记录；每条记录有相同的列数，每一列代表记录的一个属性，称之为字段。
- 在关系型数据库中，对数据的操作几乎全部建立在一个或多个关系表格上，通过对表格选择、合并、分类或连接等运算来实现数据库的管理。

非关系型数据库

- 非关系型数据库通常以键值对形式存储数据，结构不固定，每条记录可以有不一样的字段，这样可以节约存储空间。
- 非关系型数据库包括：键值存储（key-value）数据库、列存储（column-oriented）数据库、面向文档（document-oriented）数据库和图形数据库。
- 常用的 redis 是一个高性能的键值存储数据库系统。它支持存储的值类型包括字符串（string）、列表（list）、集合（set）、有序集合（zset）和哈希类型（hash）。

关系型数据库VS非关系型数据库

• 哪个比较好?

对比维度	关系型数据库 (SQL)	非关系型数据库 (NoSQL)
核心概念	数据库 → 表 → 行/记录	数据库 → 集合 → 文档
数据结构	固定模式：像表格，列必须提前定义好。	灵活模式：像JSON对象，文档结构可以随时变化。
查询语言	SQL (结构化查询语言) 标准统一，如 <code>SELECT * FROM users WHERE age > 18</code>	各家的API或专属语法 不统一，比如MongoDB的查询看起来像一个Python字典
事务支持	强：转账这类操作能保证绝对正确（ACID属性）。	较弱：追求极致性能和高并发，可能会短暂出现数据不一致（但最终会一致）。
扩展方式	纵向扩展：花钱升级硬件（换更好的CPU、加内存）。	横向扩展：花钱加机器（从1台电脑变成10台普通电脑一起工作）。
擅长场景	结构稳定、需要精确查询和关联的数据（绝大多数应用的核心：用户、订单、产品）。	海量数据、高并发读写、结构多变的数据（日志、缓存、社交关系）。

课堂小练习：猜猜我是谁

快速判断应该用关系型还是非关系型数据库：

- “我有一个微博，每天发几万条帖子” → ? 型
- “银行要存你的存款信息，一分钱都不能错” → ? 型
- “我要做一个购物车，商品数据随时变化” → ? 型

SQL 语言

- SQL 指结构化查询语言 (structured query language)，是一种 ANSI (American National Standards Institute, 美国国家标准化组织) 标准的计算机语言。
- 管理和组织关系数据库中的数据，需要使用 SQL 语言。它是关系数据库的通用语言。
- 不同的数据库会有自己增加的扩展。

SQL 语言

● SQL 中有三种类型的语言：

- **数据定义语言**（**DDL**, data definition language），用于定义数据库的用户、表、关系、约束和权限等。
- **数据操作语言**（**DML**, data manipulation language），用于实现对数据的查询、插入、更新和删除操作。
- **数据库控制语言**（**DCL**, data control language），用于授权和角色控制，例如通过 GRANT 授权，通过 REVOKE 取消授权。

SQL 语言

- 数据定义的 DDL 命令如表所示：

常用 DDL 命令

命令	说明	示例
CREATE DATABASE	创建数据库	CREATE DATABASE stockdb
DROP DATABASE	删除数据库	DROP DATABASE stockdb
CREATE TABLE	创建表格	CREATE TABLE stocks (code text, name text, price real)
DROP TABLE	删除表格	DROP TABLE stocks

SQL 语言

- 数据操作语言的 DML 命令如表所示:

常用 DML 命令

命令	说明	示例
SELECT	查询数据	SELECT * FROM stocks SELECT code,name from stocks SELECT * from stocks WHERE code='601988'
INSERT INTO	插入数据	INSERT INTO stocks VAUES('601988',' 中国银行 ', 3.19)
UPDATE	修改数据	UPDATE stocks SET price=3.29 WHERE code='601988'
DELETE FROM	删除数据	DELETE FROM stocks WHERE code='601988'



10.2

SQLITE 数据库

SQLite 数据库简介

- SQLite 是轻量级、基于磁盘文件的关系型数据库。允许使用 SQL 查询语言。
- Python 中内置了 sqlite3 模块，在访问和操作 SQLite 数据时，需要首先导入 sqlite3 模块，然后创建一个与数据库关联的 **Connection** 对象。

```
>>> import sqlite3
>>> conn = sqlite3.connect("example.db")
>>> print(conn)
```

<sqlite3.Connection at 0x7fc4e0047810>

0x7fc4e0047810 是这个对象在计算机内存中的地址（十六进制表示）

就像你办了一张图书馆借书卡（连接对象），print(卡)不会打印出里面的借书记录，而是告诉你“这是一张图书馆借书卡，编号是xxx”。

部分	含义
sqlite3	使用刚才导入的数据库模块
.connect()	调用模块里的“连接”函数
"example.db"	数据库文件的文件名
conn =	把连接结果存到变量conn里

SQLite 数据库简介

- 创建了 Connection 对象后，就可以创建 **Cursor** 对象，并调用 Cursor 对象的 `execute()` 方法来执行 SQL 语句。

```
c = conn.cursor()
c.execute("CREATE TABLE trans (date text, flag text, symbol text, qty real,price real)")
c.execute("INSERT INTO trans VALUES('2020-09-10', 'BUY', '中国银行', 100, 3.19)")
for row in c.execute("SELECT * FROM trans"):
    print(row)
```

```
('2020-09-10', 'BUY', '中国银行', 100.0, 3.19)
```

```
conn.commit()
conn.close()
```

类型	对应 Python 类型	说 明
null	NoneType	空值
integer	int	整数类型
real	float	浮点数类型
text	str	字符串类型
BLOB	bytes	二进制数据

- 在 `sqlite3` 中，支持的数据类型如表所示。

建立连接

- 使用数据库的第一步，就是建立与数据库的连接。
- 通过 sqlite3 的 connect() 方法可以创建 Connection 对象。该方法的参数是需要连接的数据库文件，例如：

```
>>> import sqlite3
>>> conn =
>>> sqlite3.connect("example.db")
print(conn)
<sqlite3.Connection at 0x7fc4e0047810>
```

如果 example.db 文件不存在，则在当前工作目录下创建该文件。

建立连接

● Connection 对象的主要方法有：

- cursor() 返回连接的游标，用于访问和操作数据库中的数据。
- commit() 方法提交当前事务，如果不提交，则从其他数据库连接中看不到自上次调用 commit() 方法以来的任何插入、删除和更新数据的操作，当前修改也不会保存到数据库文件中。
- rollback() 方法撤销当前事务，将数据库恢复至上次调用 commit() 方法后的状态。
- close() 方法关闭数据库连接，释放系统资源。

操作数据

- 在建立连接后，通过 Cursor 对象可以将操作数据的 SQL 语句发送给数据库，得到相应的结果。

- `execute()` 方法，用于执行一条 SQL 语句：

```
execute(sql [, parameters])
```

当 SQL 语句中需要使用变量值时，可以使用问号或命名变量作为占位符两种方式进行值的传递。

操作数据

【例】 利用execute()方法创建表stocks、插入并查询数据。

```
import sqlite3
conn = sqlite3.connect("example.db")
cursor = conn.cursor()
cursor.execute("CREATE TABLE stocks (code text, name text, price real)")
s_code, s_name, s_price = "601988", "中国银行", 3.19
❶ cursor.execute("INSERT INTO stocks VALUES (?, ?, ?)", (s_code, s_name, s_price))
conn.commit()
❷ result = cursor.execute("SELECT * FROM stocks WHERE code =:1 AND name=:2 ", {"1": s_code,
"2": s_name})
for r in result:
    print(r)
cursor.close()
conn.close()
```

('601988', '中国银行', 3.19)

操作数据

- `executemany()` 方法，该方法针对所有参数序列或序列的映射执行同一个 SQL 命令，第二个参数通常是一个可迭代对象。

```
executemany(sql [, parameters])
```

【例】 利用 `executemany()` 方法将多条记录插入到表 `stocks` 并查询相应数据。

```
import sqlite3
conn = sqlite3.connect("example.db")
cursor = conn.cursor()
❶ stock_tuple = (("601288", "农业银行", 3.16),
                  ("601398", "工商银行", 4.94),
                  ("601939", "建设银行", 6.19))
❷ cursor.executemany("INSERT INTO stocks VALUES (?, ?, ?)", stock_tuple)
conn.commit()
result = cursor.execute("SELECT * FROM stocks")
for r in result:
    print(r)
cursor.close()
conn.close()
```

```
('601988', '中国银行', 3.19)
('601288', '农业银行', 3.16)
('601398', '工商银行', 4.94)
('601939', '建设银行', 6.19)
```

操作数据

- `fetchone()` 方法，获取查询结果集合的下一行，返回该行字段构成的元组。如果没有更多数据可用，则返回 `None`。

`fetchone()`

【例】利用 `fetchone()` 方法查询表 `stocks` 中相应数据。

```
import sqlite3
conn = sqlite3.connect("example.db")
cursor = conn.cursor()
❶ cursor.execute("SELECT * FROM stocks")
❷ print(cursor.fetchone())
❸ print(cursor.fetchone())
❹ print(type(cursor.fetchone()))
cursor.close()
conn.close()
```

```
('601988', '中国银行', 3.19)
('601288', '农业银行', 3.16)
<class 'tuple'>
```

操作数据

- `fetchmany()` 和 `fetchall()` 方法，这两个方法也是查询数据，`fetchmany()` 方法获取查询结果的下一行开始的多行，并返回一个列表。`fetchall()` 方法获取查询结果的所有（剩余）行，并返回一个列表。

`fetchmany([size=cursor.arraysize])`

`fetchall()`

【例】利用 `fetchmany()` 和 `fetchall()` 方法查询表 `stocks` 中相应数据。

```
import sqlite3
conn = sqlite3.connect("example.db")
cursor = conn.cursor()
❶ cursor.execute("SELECT * FROM stocks")
❷ print(cursor.fetchmany(2))
❸ print(cursor.fetchall())
cursor.close()
conn.close()
```

```
[('601988', '中国银行', 3.19), ('601288', '农业银行', 3.16)]
[('601398', '工商银行', 4.94), ('601939', '建设银行', 6.19)]
```

优化查询结果

- 在 Python 中，还提供了用作 Connection 对象的高度优化的 Row 对象，支持按列名和索引、迭代、表示和 len() 方法进行映射访问。
- 若要将 fetchone() 等方法返回的数据类型改为 Row 对象，需要设置对应连接的 `row_factory`。

```
import sqlite3
conn = sqlite3.connect("example.db")
❶ conn.row_factory = sqlite3.Row
cursor = conn.cursor()
cursor.execute("SELECT * FROM stocks")
❷ r = cursor.fetchone()
❸ print("r 的类型: ", type(r))
❹ print("r 的 keys: ", r.keys())
❺ print("r 的第一个字段值: ", r[0])
❻ print("r 的 name: ", r["name"])
cursor.close()
conn.close()
```

r 的类型: <class 'sqlite3.Row'>
r 的 keys: ['code', 'name', 'price']
r 的第一个字段值: 601988
r 的 name: 中国银行

SQL课堂练习：代码找茬

- 源代码有三处错

```
import sqlite3
conn = sqlite3.connect("example.db")
cursor = conn.cursor()
cursor.execute("SELEC * FROM stocks")
conn.close()
```

SQL课堂练习1：学生选课管理系统

1. 导入 sqlite3 模块

```
import sqlite3
```

说明项	内容
模块名称	sqlite3 - Python内置模块，无需额外安装
作用	提供SQLite数据库的访问接口
特点	轻量级、基于文件、不需要单独的数据库服务器
适用场景	小型应用、移动应用、学习演示

SQL课堂练习：学生选课管理系统

2. 数据库介绍：school.db

```
conn = sqlite3.connect("school.db")
```

运行后，检查当前文件夹是否生成了 school.db 文件

```
import os
```

```
print("数据库文件是否存在?", os.path.exists("school.db"))
```

说明项	内容
数据库名称	school.db
文件位置	当前Python脚本所在目录
文件类型	SQLite数据库文件（二进制文件）
创建时机	第一次连接时自动创建
查看方式	可以用SQLite Browser或命令行查看

SQL课堂练习：学生选课管理系统

3. 三张数据表介绍

表1：学生表students

表2：课程表courses

表3：选课表enrollments

字段名	数据类型	约束	说明	示例值
student_id	INTEGER	PRIMARY KEY	学生学号 (主键)	1, 2, 3...
name	TEXT	NOT NULL	学生姓名	张三
class	TEXT	-	班级	软件工程1班
score	INTEGER	-	总分	85

字段名	数据类型	约束	说明	示例值
course_id	INTEGER	PRIMARY KEY	课程编号 (主键)	101, 102...
course_name	TEXT	NOT NULL	课程名称	Python程序设计
credit	INTEGER	-	学分	3

字段名	数据类型	约束	说明	示例值
student_id	INTEGER	FOREIGN KEY	学生编号 (外键)	1
course_id	INTEGER	FOREIGN KEY	课程编号 (外键)	101
semester	TEXT	-	学期	2024秋季

第一步：创建数据库和表格

```
import sqlite3
```

```
conn = sqlite3.connect("school.db")  
cursor = conn.cursor()
```

```
# 1. 学生表
```

```
cursor.execute('''  
CREATE TABLE students (  
    student_id INTEGER PRIMARY KEY,  
    name TEXT NOT NULL,  
    class TEXT,  
    score INTEGER  
)  
''')
```

```
# 2. 课程表
```

```
cursor.execute('''  
CREATE TABLE courses (  
    course_id INTEGER PRIMARY KEY,  
    course_name TEXT NOT NULL,  
    credit INTEGER  
)  
''')
```

```
# 3. 选课表 (连接学生和课程的多对多关系)
```

```
cursor.execute('''  
CREATE TABLE enrollments (  
    student_id INTEGER,  
    course_id INTEGER,  
    semester TEXT,  
    FOREIGN KEY (student_id) REFERENCES students(  
        student_id),  
    FOREIGN KEY (course_id) REFERENCES courses(co  
        urse_id)  
)  
''')
```

```
print("✅ 三张表创建成功!")
```

第二步：插入数据 (DML - INSERT)

插入学生数据

```
students_data = [
    (1, "张三", "软件工程1班", 85),
    (2, "李四", "软件工程1班", 72),
    (3, "王五", "软件工程2班", 93),
    (4, "赵六", "软件工程2班", 68),
    (5, "小明", "大数据1班", 88),
    (6, "小红", "大数据1班", 91),
]

cursor.executemany("INSERT INTO students VALUE
    S (?, ?, ?, ?)", students_data)
```

插入课程数据

```
courses_data = [
    (101, "Python程序设计", 3),
    (102, "数据库原理", 4),
    (103, "Web开发", 3),
    (104, "人工智能", 4),
]

cursor.executemany("INSERT INTO courses VALUES
    (?, ?, ?)", courses_data)
```

插入选课数据 (谁选了哪门课)

```
enrollments_data = [
    (1, 101, "2024秋季"),
    (1, 102, "2024秋季"),
    (2, 101, "2024秋季"),
    (3, 102, "2024秋季"),
    (3, 103, "2024秋季"),
    (3, 104, "2024秋季"),
    (4, 101, "2024秋季"),
    (5, 102, "2024秋季"),
    (5, 103, "2024秋季"),
    (6, 101, "2024秋季"),
    (6, 104, "2024秋季"),
]

cursor.executemany("INSERT INTO enrollments VALUE
    S (?, ?, ?)", enrollments_data)

conn.commit()
print("✅ 数据插入成功! ")
```

第三步：选择 (SELECT) - 各种查询

练习1: 查询所有学生

```
print("\n📖 全体学生列表: ")
cursor.execute("SELECT * FROM students")
for row in cursor.fetchall():
    print(f" {row}")
```

练习2: 查询成绩大于80分的学生

```
print("\n🌟 成绩大于80分的学霸: ")
cursor.execute("SELECT name, score FROM students WHERE score > 80")
for row in cursor.fetchall():
    print(f" {row[0]} - {row[1]}分")
```

练习3: 按班级分组, 显示各班平均分

```
print("\n📊 各班平均分: ")
cursor.execute('''
    SELECT class, AVG(score) as avg_score, COUNT(*) as student_count
    FROM students
    GROUP BY class
''')
for row in cursor.fetchall():
    print(f" {row[0]}: 平均分 {row[1]:.1f}, 共{row[2]}人")
```

第四步：连接 (JOIN) - 多表联合查询

4.1 内连接 - 查询每个学生选了哪些课

练习4: 查询每个学生的选课情况 (内连接)

```
print("\n📖 学生选课清单: ")
cursor.execute('''
    SELECT s.name, c.course_name, e.semester
    FROM students s
    JOIN enrollments e ON s.student_id = e.student_id
    JOIN courses c ON e.course_id = c.course_id
    ORDER BY s.name
''')

current_student = None
for name, course, semester in cursor.fetchall():
    if name != current_student:
        print(f"\n   【{name}】 选了: ")
        current_student = name
    print(f"       - {course} ({semester})")
```


第四步：连接 (JOIN) - 多表联合查询

4.2 左连接 - 找出没选课的学生

练习5：找出还没选课的学生（用LEFT JOIN）

```
print("\n🔔 还没选课的同学（需要提醒）：")
```

```
cursor.execute('''
```

```
    SELECT s.name, s.class
```

```
    FROM students s
```

```
    LEFT JOIN enrollments e ON s.student_id = e.student_id
```

```
    WHERE e.student_id IS NULL
```

```
''')
```

```
for row in cursor.fetchall():
```

```
    print(f" ⚠️ {row[0]} ({row[1]}) - 请尽快选课!")
```

第五步：分类和分组（GROUP BY + 聚合函数）

练习6：每门课程的选课人数（分类统计）

```
print("\n🏠 课程热度排行榜：")
cursor.execute('''
    SELECT c.course_name, COUNT(e.student_id) as student_count
    FROM courses c
    LEFT JOIN enrollments e ON c.course_id = e.course_id
    GROUP BY c.course_id
    ORDER BY student_count DESC
''')
for i, (course, count) in enumerate(cursor.fetchall(), 1):
    medal = "🥇" if i == 1 else "🥈" if i == 2 else "🥉" if i == 3 else " "
    print(f" {medal} {course}: {count}人选修")
```

第六步：合并 (UNION) - 合并多个查询结果

练习7：合并「高分段」和「需要加油」的学生

```
print("\n📊 成绩两极分化名单：")
```

```
cursor.execute('''
```

```
    SELECT name, score, '🔥 学霸区' as tag FROM students WHERE score >= 90
```

```
    UNION
```

```
    SELECT name, score, '📉 潜力区' as tag FROM students WHERE score <= 70
```

```
    ORDER BY score DESC
```

```
''')
```

```
for name, score, tag in cursor.fetchall():
```

```
    print(f"    {tag}: {name} ({score}分)")
```

第七步：更新和删除（UPDATE + DELETE）

练习8：给低分学生加油（加安慰分）

```
print("\n🎁 给低分学生加5分安慰分...")
```

```
cursor.execute("UPDATE students SET score = score + 5 WHERE score < 70")
```

```
conn.commit()
```

```
cursor.execute("SELECT name, score FROM students WHERE score < 75")
```

```
print(" 加分后的低分段同学：")
```

```
for name, score in cursor.fetchall():
```

```
    print(f"    {name} -> {score}分")
```

练习9：删除一个学生（演示）

```
print("\n🗑️ 演示删除：如果学生退学了...")
```

注意：实际删除前需要先删除关联的选课记录

```
cursor.execute("DELETE FROM enrollments WHERE student_id = 4")
```

```
cursor.execute("DELETE FROM students WHERE student_id = 4")
```

```
conn.commit()
```

```
cursor.execute("SELECT name FROM students")
```

```
print(" 剩余学生：", [row[0] for row in cursor.fetchall()])
```

第八步：综合应用 - 查询某个学生的详细学习报告

练习10: 【挑战题】生成学生个人学习报告

```
print("\n📄 学生个人学习报告: ")
```

```
student_name = "张三" # 可以让学生自己改名字体验
```

```
cursor.execute('''
    SELECT
        s.name,
        s.class,
        s.score,
        GROUP_CONCAT(c.course_name) as courses,
        COUNT(c.course_id) as course_count,
        SUM(c.credit) as total_credits
    FROM students s
    JOIN enrollments e ON s.student_id = e.student_id
    JOIN courses c ON e.course_id = c.course_id
    WHERE s.name = ?
    GROUP BY s.student_id
''', (student_name,))
```

```
result = cursor.fetchone()
```

```
if result:
    name, class_name, score, courses, count, credits = result
    print(f'''
```

```
    学生: {name}
    班级: {class_name}
    总成绩: {score}分
    选修课程数: {count}门
    总学分: {credits}学分
    课程列表: {courses}
```

```
''')
```

```
else:
    print(f"🔍 未找到学生: {student_name}")
```

```
conn.close()
```

```
print("✅ 数据库连接已关闭")
```

扩展挑战题

尝试完成以下任务：

- 挑战1：写查询找出「选了所有课」（选课最多的）学生
- 挑战2：写查询找出「没选数据库原理」的学生
- 挑战3：新增一门课程「数据结构」（学分4），并让王五选修它
- 挑战4：创建一个视图 student_report，包含学生的完整学习信息

扩展挑战题

期望输出：

【挑战2】🔍 找出「没选数据库原理」的学生

以下同学没有选修「数据库原理」：

- 📖 张三（软件工程1班）
- 📖 李四（软件工程1班）
- 📖 赵六（软件工程2班）
- 📖 小红（大数据1班）

【挑战3】+ 新增课程并安排王五选修

- ✅ 成功添加新课程：数据结构（ID：105，学分：4）
- ✅ 成功！王五选修了「数据结构」
- 📖 王五 现在的课程：Python程序设计，Web开发，人工智能，数据库原理，数据结构

扩展挑战题

期望输出：

【挑战4】  创建「学生完整学习报告视图」

 学生学习报告（完整版）：

学号	姓名	班级	总分	课程数	学分	等级	学习状态
3	王五	软件工程2班	93	5	18	优秀 (A)	 课程全勤王
6	小红	大数据1班	91	2	7	优秀 (A)	 正常
...							



10.3

操作其他关系型数据库

操作 MySQL 数据库

- Python 使用 pymysql 模块来访问和操作 MySQL 数据库，该模块主要方法与 sqlite3 模块中的方法类似，如下所示：

- (1) `execute()`: 执行一条 SQL 语句。
- (2) `executemany()`: 利用一条 SQL 语句，重复执行可迭代参数中的每个值。
- (3) `nextset()`: 移到下一个结果集合。
- (4) `fetchall()`: 获取返回结果的全部行。
- (5) `fetchmany()`: 获取返回结果中指定数量的行，输入参数控制返回行数。
- (6) `fetchone()`: 获取返回结果中的一行数据。
- (7) `commit()`: 提交事务。
- (8) `rollback()`: 返回事务。

操作 MySQL 数据库

【例】使用 pymysql 模块操作 MySQL 数据库的代码如下所示：

```
import pymysql
❶ conn = pymysql.connect(host="localhost", user="root", password="1234",
                           db="test", port=3306, charset="utf8")

cur = conn.cursor()
❷ cur.execute('''CREATE TABLE stocks ('code' varchar(6) NOT NULL,
                                       'name' varchar(20) NOT NULL,
                                       PRIMARY KEY ('code'))''')

cur.executemany("INSERT INTO stocks VALUES (%s, %s)",
                [("601988", "中国银行"), ("601288", "农业银行")])

conn.commit()
❸ cur.execute("SELECT * FROM stocks")
❹ print(cur.fetchone())
cur.close()
conn.close()

('601988', '中国银行')
```

操作 SQL Server 数据库

● Python 可以使用 pywin32、pyodbc、pymssql 等不同模块操作 SQL Server 数据库。这里我们使用 pymssql 模块，该模块是第三方模块，需要通过 pip 安装。在安装完成后，可以通过以下步骤操作 SQL Server 数据库：

- (1) **创建连接**：使用 `pymssql.connect()` 方法创建连接并获取 Connection 对象。
- (2) **操作数据**：通过 Connection 对象的 `cursor()` 方法获取 Cursor 对象，使用该对象的各种方法与数据库进行交互操作。
- (3) **关闭连接**。

操作 SQL Server 数据库

【例】 Python 操作 SQL Server 的方法和操作 sqlite3、MySQL 非常类似，如下：

```
❶ import pymssql
❷ server_name, user_name, password, db_name = ("127.0.0.1", "sa",
                                                "123456", "stockdb")
❸ conn = pymssql.connect(server_name, user_name, password, db_name)
   cursor = conn.cursor()
   cursor.execute('''CREATE TABLE stocks (code VARCHAR(6),
                                           name VARCHAR(100),
                                           PRIMARY KEY(code))''')
   cur.executemany("INSERT INTO stocks VALUES (%s, %s) ",
                   [("601988", "中国银行"), ("601288", "农业银行")])
   conn.commit()
❹ cursor.execute("SELECT * FROM persons WHERE code=%s", "601288")
   row = cursor.fetchone()
   print("代码: {}, 名称: {}".format(row[0], row[1]))
   cursor.close()
   conn.close()
```

('601288', '农业银行')

连接 Oracle 数据库

- 连接Oracle需要安装cx_Oracle模块和Oracle客户端 Oracle Instant Client:

```
import cx_Oracle
❶ conn = cx_Oracle.connect("sys", "1234",
    "127.0.0.1:1621/orcl")
cur = conn.cursor()
cur.execute("SELECT * FROM stocks")
data = cur.fetchone()
print(data)
cur.close()
conn.close()
```

('601288', '农业银行')



10.4

操作 MONGODB 数据库

MongoDB 数据库简介

- MongoDB 是基于分布式文件存储的数据库，是功能最丰富、最接近关系数据库的非关系数据库之一。
- MongoDB 使用 JSON 结构将其数据存储在文档中，灵活且不需要架构。
- 在大数据时代，MongoDB 是存储文本等非关系型数据的重要工具之一。
- 第三方库 pymongo 完美支持 MongoDB 数据的操作，可以使用 pip 命令安装 pymongo 模块：

```
pip install pymongo
```

```
(base) GCHEN@GarysMacBookPro ~ % pip install pymongo
Collecting pymongo
  Downloading pymongo-4.17.0-cp313-cp313-macosx_11_0_arm64.whl.metadata (10 kB)
Collecting dnspython<3.0.0,>=2.6.1 (from pymongo)
  Downloading dnspython-2.8.0-py3-none-any.whl.metadata (5.7 kB)
Downloading pymongo-4.17.0-cp313-cp313-macosx_11_0_arm64.whl (985 kB)
-----985.9/985.9 kB 1.3 MB/s eta 0:00:00
Downloading dnspython-2.8.0-py3-none-any.whl (331 kB)
Installing collected packages: dnspython, pymongo
Successfully installed dnspython-2.8.0 pymongo-4.17.0
```

使用 pymongo 连接数据库

- 使用Mongo数据之前，需首先与Mongo客户端建立连接，导入 pymongo 模块

```
>>> from pymongo import MongoClient
>>> client = MongoClient('localhost',27017)
>>> db = client.stockNews
>>> print(db.collection_names())
['GubaNews','system.indexes']

>>> gubaNews = db.GubaNews
```

MongoDB 数据库操作

- `insert_one()` 方法在集合（相当于关系数据库中的表）中插入一个文档：

```
>>> news = {'title': '以科技创新为本中国智造再造双循环势能',  
            'date': '2020年09月30日',  
            'content': '最近，双循环是中国经济的热门话题。以国内大循环为主体...'}  
gubaNews.insert_one(news)  
<pymongo.results.InsertOneResult object at 0x7f8fbe7fb960>
```

- `insert_many()` 方法是将多个文档一次性插入集合中：

```
>>> gubaNews.insert_many([{'title': '以科技创新为本中国智造再造双循环势能',  
                           'date': '2020年09月30日',  
                           'content': '最近，双循环是中国经济的热门话题。以国内大循环为主体...'},  
                           {'title': '广东将扶持海上风电发展',  
                           'date': '2020年09月30日',  
                           'content': '广东省昨日印发广东省培育新能源战略性新兴产业集群行动计划...'},  
                           {'title': '城市更新焕发老城市新活力',  
                           'date': '2020年09月30日',  
                           'content': '华灯初上，光塔街怡乐里社区的街坊纷纷出门...'}])  
<pymongo.results.InsertManyResult object at 0x7f8fbe7fb7d0>
```

MongoDB 数据库操作

- `insert()` 方法既可以插入一个文档，也可以插入多个文档。该方法返回所插入文档所有对象的 ObjectId，用来唯一标识文档：

```
>>> gubaNews.insert({'title': '以科技创新为本中国智造再造双循环势能',  
                    'date': '2020年09月30日',  
                    'content': '最近，双循环是中国经济的热门话题。以国内大循环为主体...'})  
  
ObjectId('57611d4b1aa303032ad5ba9e')  
  
>>> gubaNews.insert([{'title': '广东将扶持海上风电发展',  
                      'date': '2020年09月30日',  
                      'content': '广东省昨日印发广东省培育新能源战略性新兴产业集群行动计划...'},  
                    {'title': '城市更新焕发老城市新活力',  
                      'date': '2020年09月30日',  
                      'content': '华灯初上，光塔街怡乐里社区的街坊纷纷出门...'}])  
  
[ObjectId('57611d4b1aa303032ad5ba9e'), ObjectId('57611d4b1aa30303sdd5ba9e')]
```

MongoDB 数据库操作

- `find()` 和 `find_one()` 方法可以用来查询文档：`find()` 方法将返回该集合中符合条件的所有文档。在默认情况下，它返回一个游标对象。`find_one()` 方法返回集合中符合条件的第一个文档：

```
>>> gubaNews.find()
<pymongo.cursor.Cursor object at 0x7f8fc1853890>

>>> gubaNews.find_one()
{'date': '2020 年 09 月 30 日', 'title': '以科技创新为本中国智造再造双循环势能', '_id': ObjectId('57611d4b1aa30303lc6eba9e'), 'content': '最近，双循环是中国经济的热门话题。以国内大循环为主体...'}

>>> gubaNews.find({'title': '以科技创新为本中国智造再造双循环势能'})
<pymongo.cursor.Cursor object at 0x7f8fbe7fb960>

>>> gubaNews.find_one({'title': '以科技创新为本中国智造再造双循环势能'})
{'date': '2020 年 09 月 30 日', 'title': '以科技创新为本中国智造再造双循环势能', '_id': ObjectId('57611d4b1aa30303lc6eba9e'), 'content': '最近，双循环是中国经济的热门话题。以国内大循环为主体...'}

```

MongoDB 数据库操作

- 修改集合中文档的方法有四种，`update()`、`update_one()`、`update_many()` 和 `replace_one()`。这四种方法的整体语法规则如下：

```
<method_name> (condition, document, upsert=False, bypass_document_validation = False)
```

其中，`condition` 是与要替换的文档匹配的查询条件，`document` 为新文档，`upsert` 是可选参数，默认为 `False`，当该参数为 `True` 时，则在没有文档与过滤器匹配的情况下执行插入。`bypass_document_validation` 也是可选参数，默认为 `False`，当该参数为 `True` 时，则允许写入选择退出文档级验证。

MongoDB 数据库操作

【例】 以下是这四种方法使用的示例，除了 update() 方法，所有方法都返回 UpdateResult 对象。

```
>>> gubaNews.update_one({"title": "城市更新焕发老城市新活力"},  
                        {"$set": {"content": "内容示例"}})  
<pymongo.results.UpdateResult object at 0x7f8fbe7fb910>  
  
>>> gubaNews.update_many({"title": "城市更新焕发老城市新活力"},  
                          {"$set": {"content": "内容示例"}})  
<pymongo.results.UpdateResult object at 0x7f8fbe7fb7d0>  
  
>>> gubaNews.update({"title": "城市更新焕发老城市新活力"},  
                    {"$set": {"content": "内容示例"}})  
{'updatedExisting': True, 'nModified': 0, 'ok': 1, 'n': 1}  
  
>>> gubaNews.replace_one({"title": "城市更新焕发老城市新活力"},  
                          {"title": "城市换新颜"})  
<pymongo.results.UpdateResult object at 0x7f8fbe7fb910>
```

MongoDB 数据库操作

- `delete_one()`和`delete_many()`方法用于删除文档操作。添加`condition`参数，可以按照指定条件删除文档，最终返回 `DeleteResult` 对象。另外，`remove()` 方法也可以实现删除文档操作，并返回删除状态。

```
>>> gubaNews.delete_one({"title":"城市更新焕发老城市新活力"})
<pymongo.results.DeleteResult object at 0x7f8fbe7fba00>

>>> gubaNews.delete_many({"title":"广东将扶持海上风电发展"})
<pymongo.results.DeleteResult object at 0x7f8fbe7fb960>

>>> gubaNews.remove({"title":"以科技创新为本中国智造再造双循环势能"})
{'ok': 1, 'n': 1}
```


Happy coding! 🐍



综合练习2：图书借阅管理系统

- 某学校图书馆需要开发一个图书借阅管理系统。请你基于 SQL 知识，完成以下任务。
- **系统需求：**
- 记录图书信息（书名、作者、分类、库存数量）
- 记录读者信息（学号、姓名、年级、可借数量）
- 记录借阅信息（谁、借了什么书、借书日期、还书日期）

基础练习：

- 根据上述需求，设计数据库表格
- 测试插入至少4本图书、4个读者、4条借阅记录
- 查询一下信息
 - 所有科幻类图书
 - 《三体》这本书的当前库存数量
 - 每个读者当前借了多少本书（未归还）
 - 借书最多的读者姓名和借书数量
 - 没有被借阅过的图书
- 实现功能（参考框架）

附加练习：

- 生成每个读者的「借阅报告」
- 找出「最受欢迎的图书Top 3」